

Họ và tên : Trần Mạnh Hùng

MSSV: 23127195

1. Các câu hỏi mở rộng

Câu 1: Nếu người dùng tự tạo combo riêng thì mở rộng hệ thống thế nào?

Để người dùng tự tạo combo (ví dụ: chọn 1 món mặn bất kỳ + 1 nước bất kỳ để được giảm giá), ta có thể sử dụng Builder Pattern cho chính cái Combo đó.

- Tạo class ComboBuilder.
- Có các hàm addMainDish(FoodItem), addDrink(FoodItem).
- Khi build() sẽ trả về một object Combo chứa danh sách các món và tự động tính lại giá khuyến mãi.
- Lúc này Abstract Factory dùng cho các combo cố định của quán, còn Builder dùng cho combo tự chọn của khách.

Câu 2: Nếu một số món thay đổi thành phần theo mùa thì sửa Builder hay Factory?

Câu trả lời phụ thuộc vào mức độ thay đổi:

- Nếu thay đổi cấu trúc món (Quy trình tạo): Ví dụ: Mùa đông Pizza cần thêm bước "Phủ sốt nóng" mà mùa hè không có. Ta cần sửa hoặc tạo mới Builder (hoặc Director trong mẫu Builder chuẩn) để thay đổi các bước lắp ráp.
- Nếu chỉ thay đổi nguyên liệu mặc định: Ví dụ: Mùa hè Factory tạo "Pizza Hải Sản", mùa đông Factory tạo "Pizza Bò". Ta sửa Factory Method (hoặc tạo subclass WinterPizzaFactory) để trả về đối tượng với cấu hình (Builder) khác nhau.

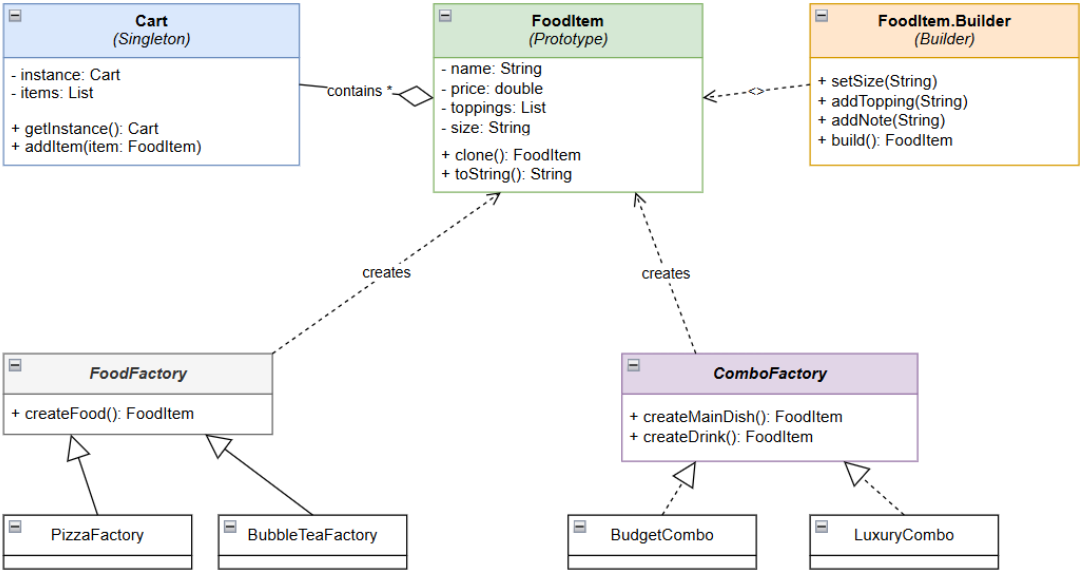
Câu 3: Nếu không dùng pattern mà dùng if/else thì có bất lợi gì?

Nếu dùng if/else (ví dụ: if (type == "PIZZA") return new Pizza(...)):

1. Vi phạm Open/Closed Principle: Mỗi lần quán thêm món mới (ví dụ: Sushi), bạn phải mở file code cũ ra, tìm đoạn if/else để chèn thêm điều kiện. Rất dễ gây lỗi cho các code đang chạy ổn định.

- 2. Code rối rắm (Spaghetti code): Khi món ăn có quá nhiều logic khởi tạo (Pizza cần nướng, Trà sữa cần lắc...), đoạn if đó sẽ dài hàng trăm dòng, cực kỳ khó đọc và bảo trì.
- 3. Khó tái sử dụng: Logic tạo Pizza bị dính chặt vào hàm if đó, nơi khác muốn tạo Pizza lại phải copy-paste code hoặc gọi hàm với hàng tá tham số null.

2. Sơ đồ UML



3. Giải thích lựa chọn Pattern và vai trò

Tên Pattern	Vai trò trong hệ thống Food	Tại sao chọn nó? (Vấn đề giải quyết)
Builder	Thợ xây món ăn	Vấn đề: Một món ăn có quá nhiều tùy chọn (Size, Đường, Đá, Topping 1, Topping 2...). Giải pháp: Thay vì viết constructor dài ngoằng <code>new TraSua("L", "50%", "30%", true, false...)</code>, Builder giúp code dễ đọc: <code>.setSize("L").addTopping("Trân châu")</code>.
Factory Method	Nhà máy sản xuất đơn lẻ	Vấn đề: Hệ thống cần tạo Pizza, Cơm, Mì... nhưng code xử lý đơn hàng không nên biết chi tiết cách tạo từng món.
Abstract Factory	Nhà máy sản xuất theo bộ (Combo)	Vấn đề: Cần tạo Combo (Món chính + Nước + Tráng miệng) sao cho chúng "hợp rơ" nhau (Combo Trưa, Combo Tối).

		Giải pháp: Đảm bảo các món trong 1 combo luôn đi cùng nhau, tránh việc Combo Tiết kiệm lại tạo ra món Tôm Hùm đắt tiền.
Prototype	Máy photocopy món ăn	Vấn đề: Khách hàng thường đặt lại món cũ (Re-order) hoặc muốn đặt 2 ly trà sữa giống hệt nhau chỉ khác mỗi ghi chú. Giải pháp: Thay vì build lại từ đầu tốn thời gian, ta clone (sao chép) object cũ ra một bản mới rồi sửa lại chút xíu.
Singleton	Quản lý duy nhất	Vấn đề: Trong 1 phiên mua sắm, khách hàng chỉ có 1 giỏ hàng. Giải pháp: Đảm bảo dù bạn đang ở trang Pizza hay trang CƠM, khi bấm "Thêm", món ăn đều chạy về đúng 1 giỏ hàng duy nhất đó.

4.Code mô phỏng giản lược

```
import java.util.ArrayList;

import java.util.List;

// --- 1. BUILDER & PROTOTYPE PATTERN: Cấu trúc món ăn ---

class MonAn implements Cloneable {

    String ten;

    String size;

    List<String> toppings = new ArrayList<>();

    double gia;

    // Constructor private để ép dùng Builder

    private MonAn(Builder builder) {

        this.ten = builder.ten;

        this.size = builder.size;
```

```
    this.toppings = builder.toppings;

    this.gia = builder.gia;
}
```

// Prototype: Clone món ăn

@Override

```
public MonAn clone() {
    try {
        MonAn copy = (MonAn) super.clone();
        copy.toppings = new ArrayList<>(this.toppings); // Copy sâu danh sách topping
        return copy;
    } catch (CloneNotSupportedException e) { return null; }
}
```

@Override

```
public String toString() {
    return "Món: " + ten + " | Size: " + size + " | Topping: " + toppings + " | Giá: " + gia;
}
```

// Builder Class

```
public static class Builder {
    private String ten;
    private String size = "Vừa"; // Mặc định
    private List<String> toppings = new ArrayList<>();
    private double gia;

    public Builder(String ten, double gia) {
        this.ten = ten;
    }
}
```

```

        this.gia = gia;
    }

    public Builder doiSize(String size) {
        this.size = size;
        return this;
    }

    public Builder themTopping(String tp) {
        this.toppings.add(tp);
        this.gia += 5000; // Thêm topping tốn 5k
        return this;
    }

    public MonAn build() {
        return new MonAn(this);
    }
}

// --- 2. FACTORY METHOD PATTERN: Nhà máy tạo món ---
class BepTruongFactory {
    // Giả lập Factory Method đơn giản
    public static MonAn lamMonAn(String loai) {
        if (loai.equals("PIZZA")) {
            return new MonAn.Builder("Pizza Phô Mai", 100000).doiSize("Lớn").build();
        } else if (loai.equals("TRASUA")) {
            return new MonAn.Builder("Trà Sữa", 30000).themTopping("Trân châu").build();
        }
    }
}

```

```

    }
    return null;
}
}

```

// --- 3. SINGLETON PATTERN: Giỏ hàng ---

```

class GioHang{
    private static GioHang instance;
    private List<MonAn> danhSachMon = new ArrayList<>();

    private GioHang() { } // Private constructor

    public static GioHang layGioHang() {
        if (instance == null) instance = new GioHang();
        return instance;
    }

    public void themVaoGio(MonAn mon) {
        danhSachMon.add(mon);

        System.out.println(">> Đã thêm: " + mon.ten + " (" + mon.size + ")");
    }

    public void thanhToan() {
        System.out.println("\n--- HÓA ĐƠN ---");

        double tongTien = 0;

        for (MonAn m : danhSachMon) {
            System.out.println(m);
            tongTien += m.gia;
        }
    }
}

```

```

    }

    System.out.println("TỔNG CỘNG: " + tongTien + " VNĐ");
}
}

// --- MAIN: CHẠY MÔ PHỎNG ---

public class AppFoodDemo {

    public static void main(String[] args) {

        // 1. Lấy giỏ hàng duy nhất (Singleton)

        GioHang cart = GioHang.layGioHang();

        System.out.println("--- KHÁCH HÀNG 1: MUA TRÀ SỮA ---");

        // 2. Dùng Factory để tạo nhanh món cơ bản

        MonAn traSuaChuan = BepTruongFactory.lamMonAn("TRASUA");

        // 3. Dùng Builder (đã ẩn trong Factory hoặc dùng trực tiếp) để custom
        // Ở đây ta giả sử khách muốn custom thêm từ món chuẩn
        // Java không hỗ trợ sửa trực tiếp Builder sau khi build, nên ta coi như tạo mới

        MonAn traSuaFullOption = new MonAn.Builder("Trà Sữa Đặc Biệt", 40000)

            .doiSize("Khổng Lồ")

            .themTopping("Pudding")

            .themTopping("Kem cheese")

            .build();

        cart.themVaoGio(traSuaChuan);    // Ly cho bạn trai

        cart.themVaoGio(traSuaFullOption); // Ly cho bạn gái
    }
}

```

```
System.out.println("\n--- KHÁCH HÀNG 1: MUỐN UỐNG THÊM LY NỮA Y HẾT LY  
FULL OPTION ---");
```

```
// 4. Dùng Prototype để clone (Sao chép)
```

```
MonAn lyThu3 = traSuaFullOption.clone();
```

```
cart.themVaoGio(lyThu3);
```

```
// In hóa đơn
```

```
cart.thanhToan();
```

```
}
```

```
}
```

Kết quả chạy mô phỏng:

```
--- KHÁCH HÀNG 1: MUA TRÀ SỮA --- >> Đã thêm: Trà Sữa (Vừa) >> Đã thêm: Trà Sữa  
Đặc Biệt (Khổng Lồ) --- KHÁCH HÀNG 1: MUỐN UỐNG THÊM LY NỮA Y HẾT LY FULL  
OPTION --- >> Đã thêm: Trà Sữa Đặc Biệt (Khổng Lồ) --- HÓA ĐƠN --- Món: Trà Sữa | Size:  
Vừa | Topping: [Trân châu] | Giá: 35000.0 Món: Trà Sữa Đặc Biệt | Size: Khổng Lồ |  
Topping: [Pudding, Kem cheese] | Giá: 50000.0 Món: Trà Sữa Đặc Biệt | Size: Khổng Lồ |  
Topping: [Pudding, Kem cheese] | Giá: 50000.0 TỔNG CỘNG: 135000.0 VNĐ
```